

Nombre del alumno:

Amador Espinoza Omar Manuel. No. Control: **1225100271**

Consumo de servicios web con REST API

Modalidad: Práctica de laboratorio, individual

1. Propósito de la actividad

Que el estudiante consuma servicios web a través de una REST API, primero con **Postman** y después con **Python** (lenguaje multiparadigma), interpretando respuestas en formato **JSON** y comparándolas con **XML**, para reafirmar los temas de la unidad:

- APIs y análisis de JSON.
- REST API utilizando Postman.
- REST API utilizando un lenguaje multiparadigma.
- Servicios web con API REST, JSON o XML.

2. Materiales

- Equipo de cómputo con acceso a Internet.
- Postman (aplicación de escritorio o web).
- Python 3 y un IDE (VS Code, PyCharm u otro).
- Librería requests (instalar con `pip install requests`).
- APIs públicas que **no requieren registro ni llave**:
 - o JSONPlaceholder: `https://jsonplaceholder.typicode.com` (API de práctica con recursos `posts`, `users`, `comments`, etc.).
 - o Open-Meteo: `https://api.open-meteo.com/v1/forecast` (pronóstico del clima).

*Nota: JSONPlaceholder **simula** las operaciones de escritura. Responde como si el POST, PUT o DELETE hubieran funcionado, pero los datos no se guardan realmente. Es normal que un GET posterior no muestre el cambio.*

3. Parte A. Repaso conceptual (20 min)

Responde en tu cuaderno o en un documento, con tus propias palabras:

- ¿Qué es una API y qué significa que una API sea REST? Es básicamente un conjunto de reglas y canales de comunicación que permiten que dos sistemas o programas de software hablen entre sí e intercambien información de manera ordenada. Y que sea REST; Significa que sigue un estilo de arquitectura (Representational State Transfer) muy popular en la web. Funciona sobre el protocolo HTTP, es ligera, no guarda estado entre una petición y otra (cada solicitud es independiente) y normalmente utiliza formatos estándar como JSON para mandar y recibir datos.
- Describe la diferencia entre cliente y servidor en una petición HTTP: Es quien hace la petición o solicita un servicio (por ejemplo, tu navegador web, la aplicación de Postman o un script de Python). Es el que inicia la comunicación preguntando por datos o enviando información. Y Servidor; es la máquina o sistema remoto que recibe esa petición, la procesa, busca la información solicitada (en bases de

datos u otros servicios) y finalmente regresa una respuesta con los datos o el resultado de la operación.

- Completa la tabla de métodos HTTP:

Método	Operación CRUD	Para qué se usa	Código de éxito típico
GET	Read (Leer)	Solicitar o consultar información al servidor sin modificarla.	200 OK
POST	Create (Crear)	Enviar nuevos datos al servidor para que cree un nuevo recurso.	201 Created
PUT	Update (Actualizar/Reemplazar)	Reemplazar por completo un recurso existente con nueva información.	200 OK (o 204 No Content)
PATCH	Update (Modificar parcialmente)	Actualizar únicamente algunos campos específicos de un recurso sin tocar los demás.	200 OK (o 204 No Content)
Método	Operación CRUD	Para qué se usa	Código de éxito típico
DELETE	Delete (Eliminar)	Borrar un recurso específico del servidor.	200 o 204 No Content

- Explica qué significan los códigos de estado: **200, 201, 204, 400, 401, 404 y 500**.
- **200 (OK)**: La petición se procesó y completó con éxito.
- **201 (Created)**: La petición fue exitosa y como resultado se creó un nuevo recurso en el servidor.
- **204 (No Content)**: La petición se procesó con éxito, pero el servidor no tiene que regresar ningún contenido en el cuerpo de la respuesta (muy común al eliminar algo).
- **400 (Bad Request)**: Error del cliente; la petición está mal armada, tiene datos inválidos o le faltan parámetros obligatorios.
- **401 (Unauthorized)**: Falta de autenticación; el usuario no ha iniciado sesión o sus credenciales son incorrectas.
- **404 (Not Found)**: El recurso que buscas no existe o la URL a la que intentas acceder es incorrecta.
- **500 (Internal Server Error)**: Hubo un fallo inesperado del lado del servidor mientras intentaba procesar la petición.

- Escribe un objeto JSON que represente a un dispositivo de red con: nombre, dirección IP, modelo, estado (activo o inactivo) y una lista de interfaces. Indica qué tipo de dato es cada valor.

```
{
  "nombre": "Switch-Core",
  "ip": "192.168.1.254",
  "modelo": "Cisco Catalyst 2960",
  "activo": true
}
```

Tipo de dato:

- **nombre:** String (Texto)
- **ip:** String (Texto)
- **modelo:** String (Texto)
- **activo:** Boolean (Verdadero/Falso)

4. Parte B. Consumo de la API con Postman (60 min)

Crema una **colección** en Postman llamada `Unidad2_Act2_TuNombre` y dentro de ella una petición por cada ejercicio. Toma captura de pantalla de la petición y de la respuesta (cuerpo, código de estado y encabezados).

#	Petición	URL	Qué debes observar
B1	GET lista de publicaciones	<code>https://jsonplaceholder.typicode.com/posts</code>	Código 200 y que el cuerpo es una lista (arreglo) de objetos.
B2	GET un recurso	<code>https://jsonplaceholder.typicode.com/posts/1</code>	Campos <code>userId</code> <code>id</code> , <code>title</code> , <code>body</code> .
B3	GET con parámetros	<code>https://jsonplaceholder.typicode.com/posts?userId=3</code>	Solo regresa publicaciones del usuario 3. Anota cuántas son.
B4	GET recurso anidado	<code>https://jsonplaceholder.typicode.com/posts/1/comments</code>	Comentarios de la publicación 1.
B5	POST crear	<code>https://jsonplaceholder.typicode.com/posts</code>	En <i>Body</i> → <i>raw</i> → <i>JSON</i> envía un objeto con <code>title</code> , <code>body</code> y <code>userId</code> . Código 201 y un <code>id</code> nuevo.
B6	PUT reemplazar	<code>https://jsonplaceholder.typicode.com/posts/1</code>	Envía un objeto completo. Compara con B2.
B7	PATCH modificar	<code>https://jsonplaceholder.typicode.com/posts/1</code>	Envía solo <code>{"title": "Nuevo título"}</code> .
B8	DELETE eliminar	<code>https://jsonplaceholder.typicode.com/posts/1</code>	Código 200 y cuerpo vacío <code>{}</code> .
B9	GET recurso inexistente	<code>https://jsonplaceholder.typicode.com/posts/9999</code>	Código 404. Explica qué significa.
B10	GET con parámetros de consulta	<code>https://api.open-meteo.com/v1/forecast?latitude=20.59&longitude=-</code>	Temperatura y viento actuales de Querétaro.

#	Petición	URL	Qué debes observar
		100.39¤t=temperature_2m,wind_speed_10m	Identifica las unidades en el campo current_units.

Preguntas de la Parte B

- ¿Qué diferencia observaste entre PUT (B6) y PATCH (B7) en lo que se envía?
PUT: Se utiliza para reemplazar o sobrescribir el recurso completo. Por ello, en la petición se debe enviar la estructura con todos los campos del objeto. **PATCH:** Se utiliza para realizar modificaciones parciales. Únicamente se envía el campo específico que se desea cambiar (en este caso, {"title": "Nuevo título"}), manteniendo intacto el resto del recurso.
- ¿Qué encabezado Content-Type regresa el servidor y qué indica?
El servidor devuelve el encabezado Content-Type: application/json; charset=utf-8.
Qué indica: Notifica al cliente que el contenido del cuerpo de la respuesta está estructurado en formato **JSON** y codificado bajo la norma de caracteres **UTF-8** (lo que garantiza la interpretación correcta de acentos y caracteres especiales).
- En B10, ¿qué parte de la URL es la ruta y cuál son los parámetros de consulta (query params)?
Ruta (Path): https://api.open-meteo.com/v1/forecast (corresponde a la dirección del servidor y el punto de acceso al servicio). **Parámetros de consulta (Query params):** Toda la sección situada después del signo ?:
 - latitude=20.59
 - longitude=-100.39
 - current=temperature_2m,wind_speed_10m

Petición	Pruebas						
GET lista de publicaciones	o's Workspace Q Search (Ctrl+K) Invite Upgrade People in the workspace omar manuel amador espinoza (...) GET New Request + No E Unidad2_Act2_AmadorEspinoza > New Request Save GEThttps://jsonplaceholder.typicode.com/posts DocsParamsAuthorizationHeaders 6BodyScriptsSettings Query Params <table><tr><th>Key</th><th>Value</th><th>Description</th></tr><tr><td>Key</td><td>Value</td><td>Description</td></tr></table> BodyCookiesHeaders 22Test Results 200 OK • 338 ms • 7.76 KB { } JSONPreviewVisualize <pre>1 [2 { 3 "userId": 1, 4 "id": 1, 5 "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit", 6 "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto" 7 }, 8 { 9 "userId": 1, 10 "id": 2, 11 "title": "qui est esse"</pre> No anomalies detected GlobalsVal	Key	Value	Description	Key	Value	Description
Key	Value	Description					
Key	Value	Description					

GET un
recurso

Workspace Q Search Invite Upgrade No Er

Overview GET Untitled Request GET Untitled Request People in the workspace GET New Request + No Er

Unidad2_Act2_AmadorEspinoza > **New Request** Save

GET ▼ https://jsonplaceholder.typicode.com/posts/1

Docs Params Authorization Headers 6 Body Scripts Settings

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers 22 Test Results 200 OK 369 ms 1.08 KB Save

{} JSON Preview Visualize

```
1 {
2   "userId": 1,
3   "id": 1,
4   "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
5   "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam
6   rerum est autem sunt rem eveniet architecto"
```

✦ No anomalies detected Globals Vaul

GET con
paámetro
s

Workspace Search Invite Upgrade

Overview GET Untitled Reque GET Untitled Reque GET Untitled Reque People in the workspace omar manuel amador espinoza (...)

Unidad2_Act2_AmadorEspinoza > New Request

GET https://jsonplaceholder.typicode.com/posts?userId=3

Docs Params Authorization Headers 6 Body Scripts Settings

Query Params

Key	Value	Description
userId	3	

Body Cookies Headers 22 Test Results 200 OK • 465 ms • 1.86 KB

{ } JSON Preview Visualize

```
1 [
2   {
3     "userId": 3,
4     "id": 21,
5     "title": "asperiores ea ipsam voluptatibus modi minima quia sint",
6     "body": "repellat aliquid praesentium dolorem quo\u00nsed totam minus non itaque\u00nnihil labore molestiae sunt dolor
7       hic recusandae veniam\u00ntempora et tenetur expedita sunt"
8   },
9   {
10    "userId": 3,
11    "id": 22,
12    "title": "dolor sint quo a velit explicando quia nec"
```

GET con
recurso
anidado

Workspace Search Invite Upgrade

Overview GET Untitled Reque GET Untitled Reque GET Untitled Reque People in the workspace omar manuel amador espinoza (...)

Unidad2_Act2_AmadorEspinoza > New Request

GET https://jsonplaceholder.typicode.com/posts/1/comments

Docs Params Authorization Headers 6 Body Scripts Settings

Query Params

Key	Value	Description

Body Cookies Headers 22 Test Results 200 OK • 300 ms • 1.53 KB

{ } JSON Preview Visualize

```
1 [
2   {
3     "postId": 1,
4     "id": 1,
5     "name": "id labore ex et quam laborum",
6     "email": "Eliseo@gardner.biz",
7     "body": "laudantium enim quasi est quidem magnam voluptate ipsam eos\u00ntempora quo necessitatibus\u00ndolor quam aute
8       quasi\u00nreiciendis et nam sapiente accusantium"
9   },
10  {
11    "postId": 1,
12    "id": 2
```

POST
crear

Workspace Search Invite Upgrade People in the workspace omar manuel amador espinoza (...)

Overview GET Untitled F GET Untitled R GET Untitled R GET New Re POST New F + No En

Unidad2_Act2_AmadorEspinoza > New Request Save

POST https://jsonplaceholder.typicode.com/posts

Docs Params Authorization Headers 7 Body Scripts Settings

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers 23 Test Results 201 Created 395 ms 1.02 KB eq. Sa

{ } JSON Preview Visualize

```
1 {
2   "id": 101
3 }
```

PUT
reemplazar

Workspace Search Invite Upgrade People in the workspace omar manuel amador espinoza (...)

Overview GET Untitled GET Untitled GET Untitled GET New Re POST New PUT New + No En

Unidad2_Act2_AmadorEspinoza > New Request Save

PUT https://jsonplaceholder.typicode.com/posts/1

Docs Params Authorization Headers 7 Body Scripts Settings

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers 21 Test Results 200 OK 524 ms 914 B eq. Sav

{ } JSON Preview Visualize

```
1 {
2   "id": 1
3 }
```

PATCH
modificar

Workspace Search Invite Upgrade People in the workspace omar manuel amador espinoza (...) PUT New PATCH New + No En

Unidad2_Act2_AmadorEspinoza > New Request

PATCH https://jsonplaceholder.typicode.com/posts/1 Save

Docs Params Authorization Headers 7 Body Scripts Settings

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers 21 Test Results 200 OK 382 ms 1.06 KB Save

{} JSON Preview Visualize

```
1 {
2   "userId": 1,
3   "id": 1,
4   "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
5   "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\n    rerum est autem sunt rem eveniet architecto"
6 }
```

DELETE
eliminar

Workspace Search Invite Upgrade People in the workspace omar manuel amador espinoza (...) PATCH New DEL New + No En

Unidad2_Act2_AmadorEspinoza > New Request

DELETE https://jsonplaceholder.typicode.com/posts/1 Save

Docs Params Authorization Headers 6 Body Scripts Settings

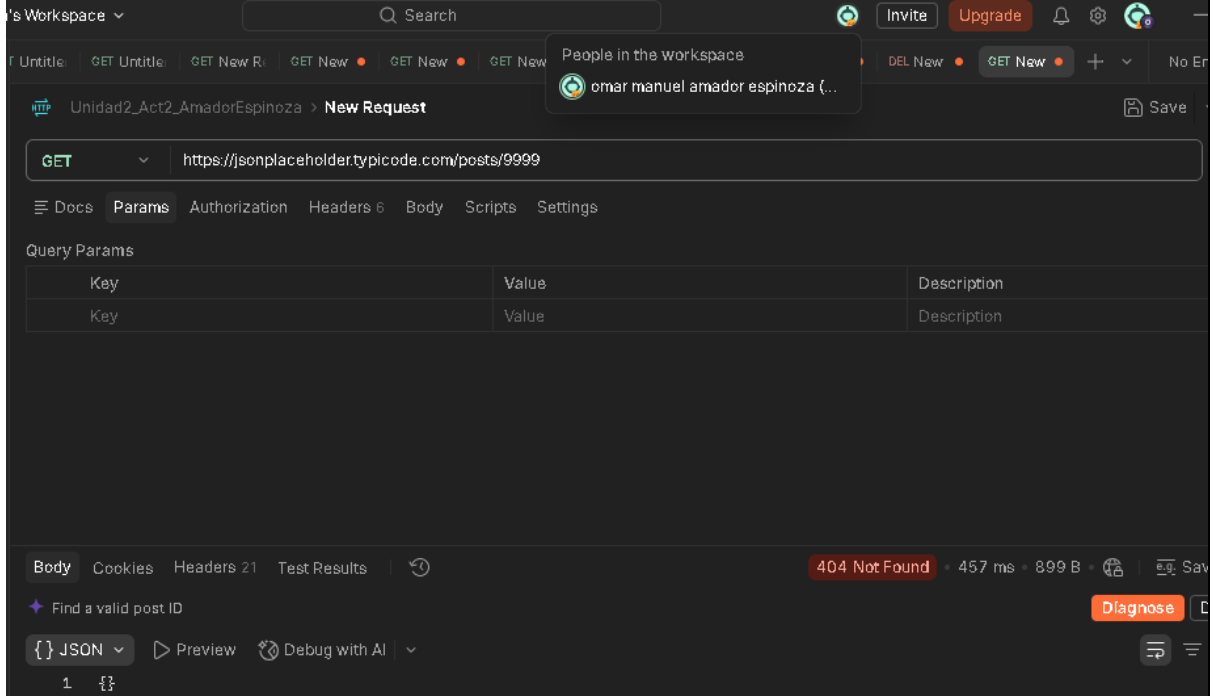
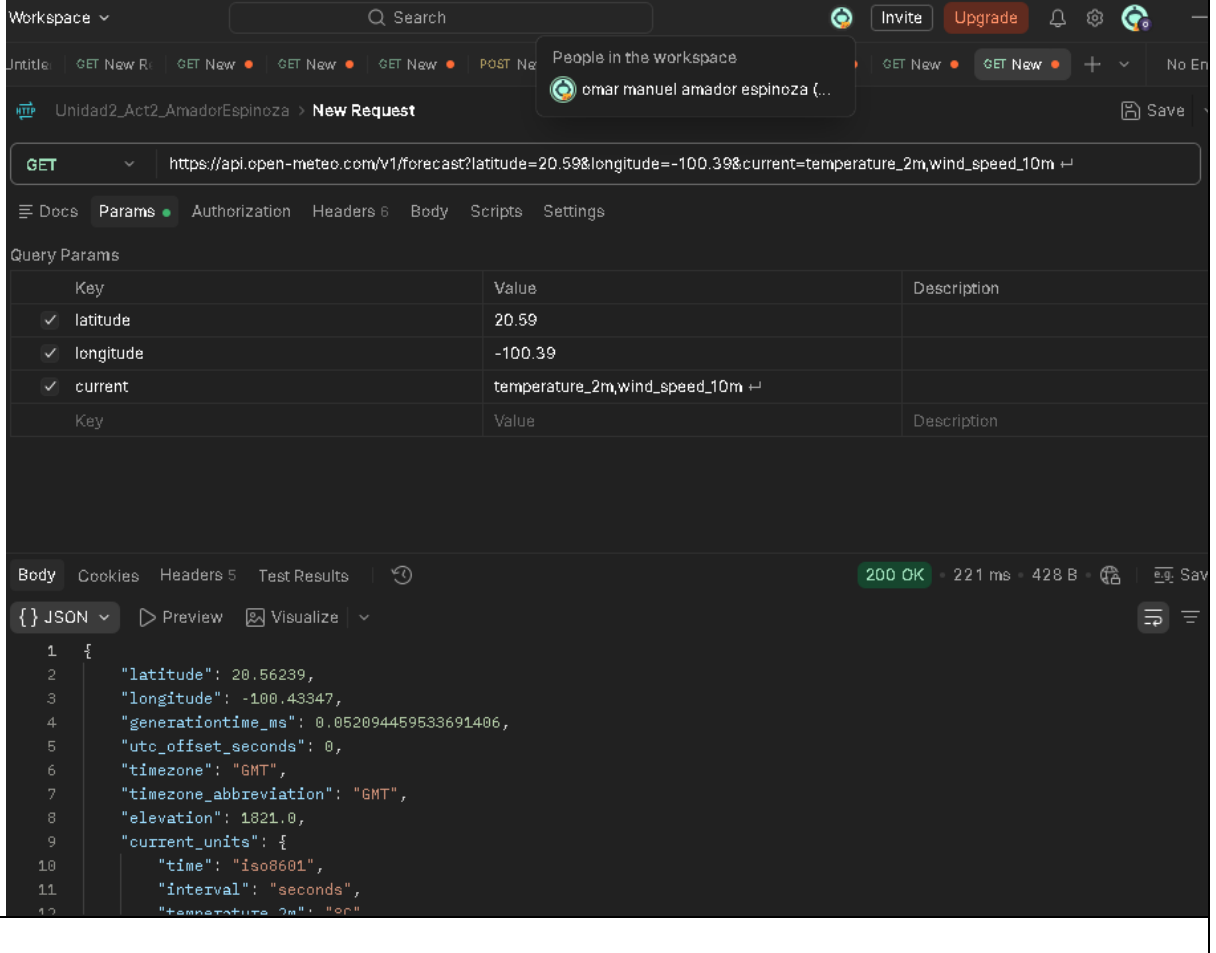
Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers 21 Test Results 200 OK 233 ms 906 B Save

{} JSON Preview Visualize

```
1 {}
```

GET recurso inexistente	 The screenshot shows a Postman interface with a GET request to <code>https://jsonplaceholder.typicode.com/posts/9999</code>. The response is a 404 Not Found error with a status of 404, a response time of 457 ms, and a body size of 899 B. The response body is empty.
GET con parámetros de consulta	 The screenshot shows a Postman interface with a GET request to <code>https://api.open-meteo.com/v1/forecast?latitude=20.59&longitude=-100.39&current=temperature_2m,wind_speed_10m</code>. The response is a 200 OK status with a response time of 221 ms and a body size of 428 B. The response body is a JSON object: <pre> 1 { 2 "latitude": 20.56239, 3 "longitude": -100.43347, 4 "generationtime_ms": 0.052094459533691406, 5 "utc_offset_seconds": 0, 6 "timezone": "GMT", 7 "timezone_abbreviation": "GMT", 8 "elevation": 1821.0, 9 "current_units": { 10 "time": "iso8601", 11 "interval": "seconds", 12 "temperature_2m": "°C" </pre>

5. Parte C. Consumo de la API con Python

Crea una carpeta `actividad_rest` y un archivo `cliente_rest.py` por cada ejercicio

C1. Primer GET

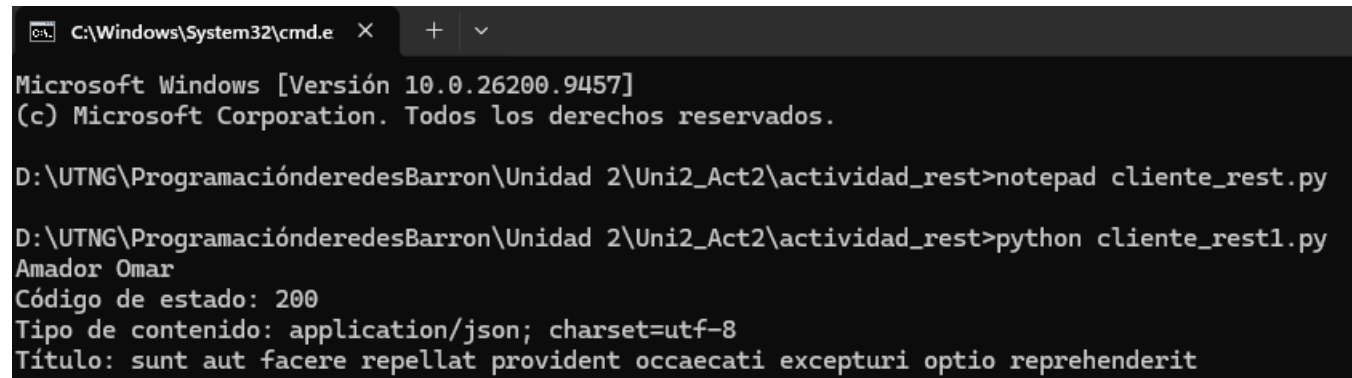
```
import requests

BASE = "https://jsonplaceholder.typicode.com"

respuesta = requests.get(f"{BASE}/posts/1", timeout=10)

print("Código de estado:", respuesta.status_code)
print("Tipo de contenido:", respuesta.headers["Content-Type"])

publicacion = respuesta.json() # convierte el JSON en un diccionario de Python
print("Título:", publicacion["title"])
```



```
C:\Windows\System32\cmd.e X + v
Microsoft Windows [Versión 10.0.26200.9457]
(c) Microsoft Corporation. Todos los derechos reservados.

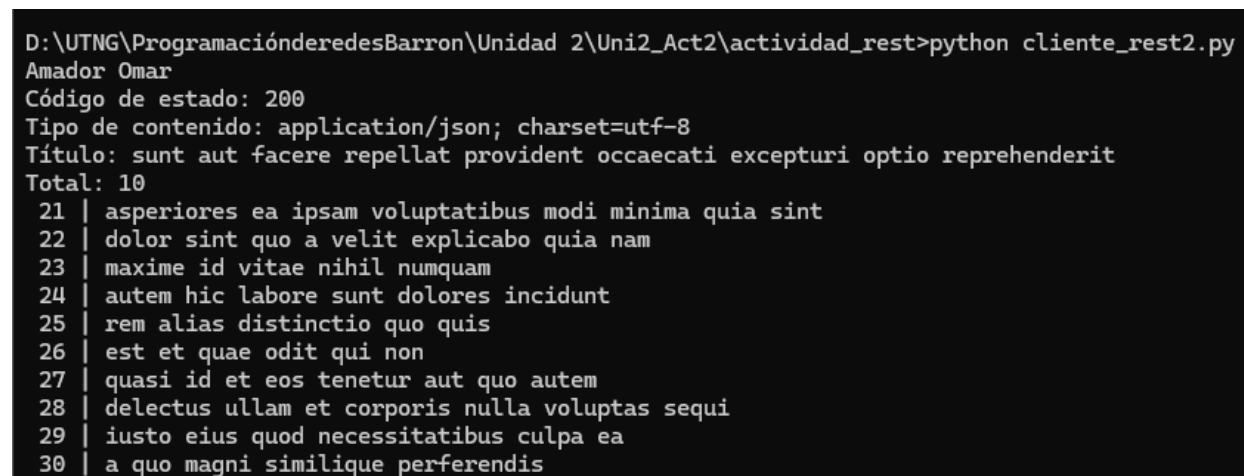
D:\UTNG\ProgramaciónredesBarron\Unidad 2\Uni2_Act2\actividad_rest>notepad cliente_rest.py

D:\UTNG\ProgramaciónredesBarron\Unidad 2\Uni2_Act2\actividad_rest>python cliente_rest1.py
Amador Omar
Código de estado: 200
Tipo de contenido: application/json; charset=utf-8
Título: sunt aut facere repellat provident occaecati excepturi optio reprehenderit
```

C2. Recorrer una lista y usar parámetros

```
respuesta = requests.get(f"{BASE}/posts", params={"userId": 3}, timeout=10)
publicaciones = respuesta.json() # lista de diccionarios

print("Total:", len(publicaciones))
for p in publicaciones:
    print(f"{p['id']}>3} | {p['title']}")
```



```
D:\UTNG\ProgramaciónredesBarron\Unidad 2\Uni2_Act2\actividad_rest>python cliente_rest2.py
Amador Omar
Código de estado: 200
Tipo de contenido: application/json; charset=utf-8
Título: sunt aut facere repellat provident occaecati excepturi optio reprehenderit
Total: 10
21 | asperiores ea ipsam voluptatibus modi minima quia sint
22 | dolor sint quo a velit explicabo quia nam
23 | maxime id vitae nihil numquam
24 | autem hic labore sunt dolores incidunt
25 | rem alias distinctio quo quis
26 | est et quae odit qui non
27 | quasi id et eos tenetur aut quo autem
28 | delectus ullam et corporis nulla voluptas sequi
29 | iusto eius quod necessitatibus culpa ea
30 | a quo magni similique perferendis
```

C3. POST, PUT y DELETE

```
nuevo = {"title": "Prueba de red", "body": "Contenido de ejemplo", "userId": 1}

r = requests.post(f"{BASE}/posts", json=nuevo, timeout=10)
print("POST:", r.status_code, r.json())

r = requests.put(f"{BASE}/posts/1", json=**nuevo, "id": 1, timeout=10)
print("PUT:", r.status_code, r.json())

r = requests.delete(f"{BASE}/posts/1", timeout=10)
print("DELETE:", r.status_code)
```

```
D:\UTNG\Programación\redesBarron\Unidad 2\Uni2_Act2\actividad_rest>python cliente_rest3.py
Amador Omar
Código de estado: 200
Tipo de contenido: application/json; charset=utf-8
Título: sunt aut facere repellat provident occaecati excepturi optio reprehenderit
Total: 10
21 | asperiores ea ipsam voluptatibus modi minima quia sint
22 | dolor sint quo a velit explicabo quia nam
23 | maxime id vitae nihil numquam
24 | autem hic labore sunt dolores incidunt
25 | rem alias distinctio quo quis
26 | est et quae odit qui non
27 | quasi id et eos tenetur aut quo autem
28 | delectus ullam et corporis nulla voluptas sequi
29 | iusto eius quod necessitatibus culpa ea
30 | a quo magni similique perferendis
POST: 201 {'title': 'Prueba de red', 'body': 'Contenido de ejemplo', 'userId': 1, 'id': 101}
PUT: 200 {'title': 'Prueba de red', 'body': 'Contenido de ejemplo', 'userId': 1, 'id': 1}
DELETE: 200
```

C4. Manejo de errores

Modifica C1 para que consulte /posts/9999 y maneje el error correctamente:

```
try:
    r = requests.get(f"{BASE}/posts/9999", timeout=10)
    r.raise_for_status()          # lanza excepción si el código es 4xx o 5xx
    print(r.json())
except requests.exceptions.HTTPError as e:
    print("Error HTTP:", e)
except requests.exceptions.RequestException as e:
    print("Error de conexión:", e)
```

```
D:\UTNG\Programación\redesBarron\Unidad 2\Uni2_Act2\actividad_rest>python cliente_rest4.py
Amador Omar
Código de estado: 200
Tipo de contenido: application/json; charset=utf-8
Título: sunt aut facere repellat provident occaecati excepturi optio reprehenderit
Total: 10
21 | asperiores ea ipsam voluptatibus modi minima quia sint
22 | dolor sint quo a velit explicabo quia nam
23 | maxime id vitae nihil numquam
24 | autem hic labore sunt dolores incidunt
25 | rem alias distinctio quo quis
26 | est et quae odit qui non
27 | quasi id et eos tenetur aut quo autem
28 | delectus ullam et corporis nulla voluptas sequi
29 | iusto eius quod necessitatibus culpa ea
30 | a quo magni similique perferendis
POST: 201 {'title': 'Prueba de red', 'body': 'Contenido de ejemplo', 'userId': 1, 'id': 101}
PUT: 200 {'title': 'Prueba de red', 'body': 'Contenido de ejemplo', 'userId': 1, 'id': 1}
DELETE: 200
Error HTTP: 404 Client Error: Not Found for url: https://jsonplaceholder.typicode.com/posts/9999
```

C5. Consumir otra API (clima)

Escribe una función `obtener_clima(latitud, longitud)` que consulte Open-Meteo y regrese un diccionario con la temperatura y la velocidad del viento. Usa `params` en lugar de armar la URL a mano.

```
def obtener_clima(latitud, longitud):
    url = "https://api.open-meteo.com/v1/forecast"
    parametros = {
        "latitude": latitud,
        "longitude": longitud,
        "current": "temperature_2m,wind_speed_10m",
    }
    r = requests.get(url, params=parametros, timeout=10)
    r.raise_for_status()
    return r.json()["current"]
```

Probarla con tres ciudades distintas (por ejemplo Querétaro: 20.59, -100.39) y mostrar los resultados en una tabla impresa en consola.

```
D:\UTNG\Programación\redesBarron\Unidad 2\Uni2_Act2\actividad_rest>python cliente_rest5.py
Amador Omar
=====
Ciudad          | Temp. (°C)   | Viento (km/h)
=====
Guadalajara     | 25.6         | 8.4
Monterrey       | 27.3         | 4.7
Cancún          | 29.5         | 8.9
=====
```

6. Parte D. JSON frente a XML (30 min)

1. Toma la respuesta JSON de `/posts/1` y escribe a mano su equivalente en XML.
2. Con Python, convierte el diccionario a XML usando `xml.etree.ElementTree`:

```
import xml.etree.ElementTree as ET

raiz = ET.Element("publicacion")
for clave, valor in publicacion.items():
    ET.SubElement(raiz, clave).text = str(valor)

print(ET.tostring(raiz, encoding="unicode"))
```

3. Completa la tabla comparativa:

Criterio	JSON	XML
Legibilidad		
Tamaño del mensaje		
Tipos de datos		
Uso común en APIs REST		

7. Parte E. Mini proyecto integrador (60 min)

Desarrolla un script `reporte_api.py` que:

1. Consulte `/users` de JSONPlaceholder y obtenga el nombre, correo y ciudad (`address.city`) de cada usuario.
2. Para cada usuario, consulte cuántas publicaciones tiene (`/posts?userId=<id>`).
3. Consulte el clima actual de un punto geográfico de tu elección.
4. Maneje errores de conexión y de código de estado.
5. Guarde el resultado en un archivo `reporte.json` (usa `json.dump` con `indent=4` y `ensure_ascii=False`) y también imprima un resumen en pantalla.

Estructura sugerida del archivo de salida:

```
{
  "clima": {"temperature_2m": 0.0, "wind_speed_10m": 0.0},
  "usuarios": [
    {"nombre": "...", "correo": "...", "ciudad": "...", "publicaciones": 0}
  ]
}
```

8. Entregables (portafolio de evidencias)

1. Documento con las respuestas de la Parte A, las preguntas de la Parte B y la tabla de la Parte D.
2. Exportación de la colección de Postman (archivo `.json`) y capturas de pantalla de B1 a B10.
3. Archivos `.py` de las Partes C y E, con comentarios.
4. Archivo `reporte.json` generado por el mini proyecto.
5. Conclusión breve (5 a 8 líneas): ¿en qué casos de administración de redes serviría consumir una API REST?

9. Lista de verificación para la evaluación

Criterio	Cumple	No cumple
Identifica correctamente métodos HTTP y códigos de estado		
Ejecuta las 10 peticiones en Postman con evidencia		
Envía cuerpo JSON y parámetros de consulta correctamente		
Consume la API con Python usando <code>requests</code>		
Convierte respuestas JSON a estructuras de Python (diccionarios y listas)		

Criterio	Cumple	No cumple
Maneja errores y tiempos de espera (timeout, raise_for_status)		
Compara JSON y XML con un ejemplo funcional		
El mini proyecto genera el archivo reporte.json correcto		
Código ordenado, comentado y entregado a tiempo		